

Card Transaction Simulator — Multithreading Learning Project

Oct 3, 2026 · @G

Tech decision: plain Java 21, no Spring Boot

Build this as a plain Java 21 Maven project with a `main()` method. No Spring, no frameworks, no AOP.

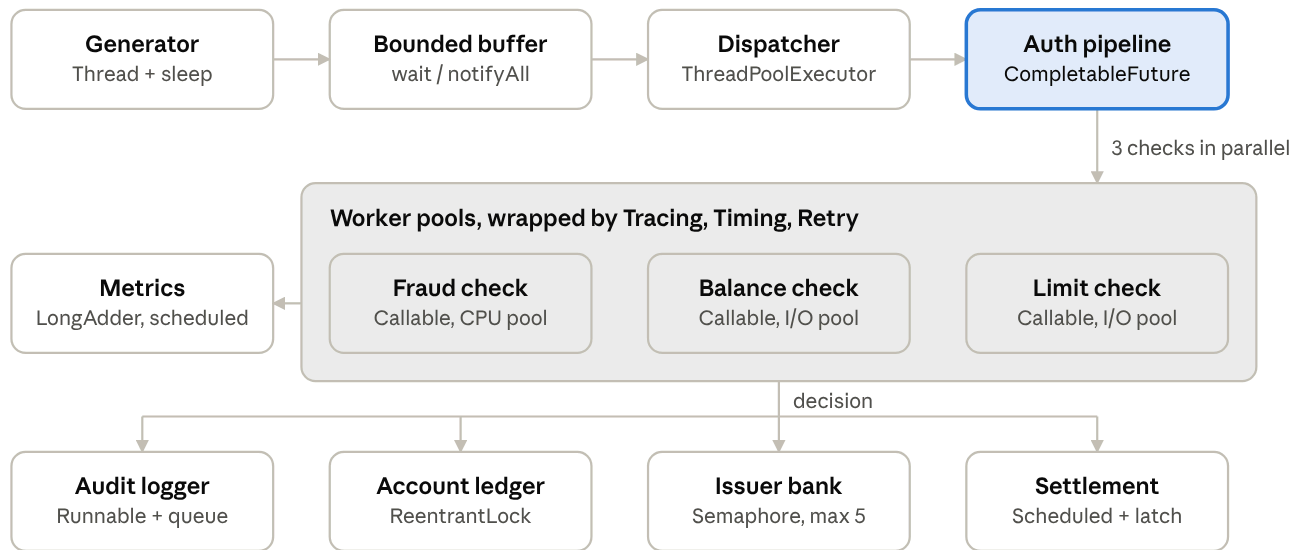
- **Why not Spring Boot:** Spring hides exactly what you want to learn. `@Async` creates executors for you, AOP wraps your tasks, and the container manages thread lifecycles. You would be configuring threading, not writing it.
- **Why Java 21:** You get virtual threads for the final comparison phase, plus records for clean immutable data objects.
- **Allowed dependencies:** SLF4J + Logback (logging with MDC) and JUnit 5. Nothing else.
- **Rule for the whole project:** every decorator, retry, timeout and metric is hand-written in Java. If you catch yourself reaching for an annotation, write the class instead.

If you later want a REST front end, you can wrap the finished engine in Spring Boot. The engine itself stays framework-free.

System overview

The simulator generates fake card transactions, authorizes each one by running three checks in parallel, and records the result in a ledger, an audit file and a periodic settlement report. Graceful shutdown (R12) wraps the whole thing.

Each transaction is buffered, checked in parallel, then recorded



simulator architecture · 12 components

Read it left to right, then down: the buffer gives backpressure, the pipeline fans out to the pools, and each decision fans out to the four recorders.

Functional requirements

Each component below is a separate class. The tag in brackets is the concept it forces you to use. Note that the audit logger (a Runnable) and the ingestion buffer (wait/notify) are two different components; the early Runnable workers simply read from that buffer.

R1. Transaction generator — the card network [Thread, Runnable, sleep]

- Runs on its own thread and produces `Transaction` records (id, cardId, accountId, amount, merchant, timestamp).
- Rate is configurable, default 50 per second; uses `Thread.sleep()` between transactions to hold that rate.
- Must stop cleanly when interrupted: catch `InterruptedException`, restore the interrupt flag, exit the loop.

R2. Ingestion buffer [synchronized, wait, notifyAll]

- Your own `BoundedBuffer<T>` class, capacity 100. No `java.util.concurrent` classes inside it.

- `put()` blocks when full (this is backpressure: a slow processor slows the generator down).
- `take()` blocks when empty.
- Must use `while` loops around `wait()`, not `if`, to survive spurious wakeups.
- Later you swap it for `ArrayBlockingQueue` behind the same interface and compare.

R3. Processing workers [Runnable first, then ExecutorService]

- Version 1: 4 raw `Thread`s, each running a `Runnable` loop: take from buffer, process, repeat.
- Version 2: one dispatcher thread takes from the buffer and submits each transaction to a fixed `ThreadPoolExecutor` you construct yourself (core size, max size, bounded queue, named `ThreadFactory`, `CallerRunsPolicy`).

R4. Check services [Callable]

- `FraudCheck`: CPU-bound, burns roughly 5–20 ms computing a score. Score above 80 means decline.
- `BalanceCheck`: simulated DB call, sleeps 50–200 ms. Fails randomly 5% of the time with a transient exception.
- `LimitCheck`: simulated cache call, sleeps 10–30 ms. Declines if daily spend plus amount exceeds the card limit.
- Each is a `Callable<CheckResult>` so it returns a result and can throw.

R5. Authorization pipeline [CompletableFuture]

- Run all three checks in parallel with `supplyAsync`, each on the right pool (fraud on the CPU pool, balance and limit on the I/O pool).
- Combine with `allOf` or `thenCombine` into one `Decision` (APPROVED or DECLINED with reason).
- Fraud check has a 300 ms timeout via `orTimeout`; on timeout, decline with reason `FRAUD_TIMEOUT`.
- Any other exception goes through `exceptionally` or `handle` to a fallback decline. No transaction is ever lost silently.
- On approval, `thenAccept` debits the ledger and sends an audit event.

R6. Account ledger [ReentrantLock, deadlock]

- Holds balances for 20 test accounts. Debits take a per-account `ReentrantLock`.
- Add a `transfer(from, to, amount)` method. Write it naively first and reproduce a deadlock with two threads transferring in opposite directions. Then fix it with lock ordering by account id, and add `tryLock` with a timeout as a second approach.

R7. Audit logger [Runnable, BlockingQueue]

- One dedicated background thread running a `Runnable` that takes audit events from a `LinkedBlockingQueue` and appends them to a file.
- Why a separate thread: file I/O must never sit on the authorization hot path. Processors just drop an event on the queue and move on.
- Stops via a poison-pill event during shutdown, after writing everything still queued.

R8. Issuer bank client [Semaphore]

- Approved transactions above 10,000 need a call to a mock issuer bank (sleeps 100 ms).
- The bank allows at most 5 concurrent calls. Enforce it with a `Semaphore`.

R9. Settlement [ScheduledExecutorService, CountdownLatch]

- Every 5 seconds a scheduled task settles the batch of approved transactions: totals per merchant, printed as a report.
- Before settling, it waits for in-flight transactions of that batch to finish, using a `CountDownLatch` (or a `Phaser` as a stretch).

R10. Metrics [AtomicLong, LongAdder, scheduled Runnable]

- Counters: received, approved, declined by reason, timed out, retried.
- A scheduled `Runnable` prints them every 2 seconds, plus pool stats (active threads, queue size).

R11. Executor decorators [Decorator pattern on Executor / Runnable / Callable]

- `TracingExecutor`: wraps any `Executor`; copies the caller's MDC (transaction id) into the worker thread and clears it afterwards, so every log line carries the transaction id.
- `TimingExecutor`: records queue-wait time and run time per task.

- `RetryingCallable<T>` : wraps a `Callable` ; retries up to 3 times with backoff (100, 200, 400 ms via `sleep`) only on transient exceptions.
- Decorators must stack: `new TracingExecutor(new TimingExecutor(pool))` .

R12. Graceful shutdown [interrupt, shutdown, awaitTermination]

- Triggered by a JVM shutdown hook or after a fixed run time (default 60 s).
- Order: stop generator, drain buffer, finish in-flight pipelines, run final settlement, stop audit logger, shut down pools with `shutdown()` then `awaitTermination()` , then `shutdownNow()` if they overrun.
- At the end, received count must equal approved plus declined. That equality is your proof nothing was lost.

Concept map: where, and why there

The "why here" column is the real lesson. In an interview, this is the reasoning they want to hear.

Concept	Used in	Why this is the right tool there
<code>Thread</code> + <code>Runnable</code>	Generator, v1 workers, audit logger	Long-running loops that return nothing. You see thread lifecycle directly before any abstraction hides it.
<code>sleep()</code>	Generator rate, mock latency, retry backoff	Pauses the current thread without releasing locks. Fine for timing, wrong for waiting on a condition.
<code>wait()</code> / <code>notifyAll()</code>	Ingestion buffer	Waiting for a condition ("buffer not full") that another thread will change. Releases the monitor while waiting, unlike <code>sleep</code> .

Concept	Used in	Why this is the right tool there
<code>BlockingQueue</code>	Audit logger, buffer v2	The library version of the wait/notify buffer. Once you've written R2 yourself, you know what it does internally.
<code>ExecutorService</code> / <code>ThreadPoolExecutor</code>	v2 workers, CPU pool, I/O pool	Reuses threads, bounds concurrency, gives a queue and a rejection policy. Separate pools stop slow I/O from starving CPU work.
<code>Callable</code>	Fraud, balance, limit checks	Needs a return value and a checked exception. <code>Runnable</code> can do neither.
<code>Future</code>	Brief step before <code>CompletableFuture</code>	Shows the pain: <code>get()</code> blocks, and combining three results is clumsy. That pain is why <code>CompletableFuture</code> exists.
<code>CompletableFuture</code>	Authorization pipeline	Runs checks in parallel and composes them without blocking a thread per step. Built-in timeout and error recovery.
Executor decorators	Tracing, timing, retry	Adds cross-cutting behaviour to every task without touching task code. This is what AOP does for you in Spring, written by hand.
<code>ReentrantLock</code> / <code>tryLock</code>	Ledger debits and transfers	Per-account locking, plus timeouts to escape deadlock, which <code>synchronized</code> cannot do.

Concept	Used in	Why this is the right tool there
<code>Semaphore</code>	Issuer bank client	Limits how many threads do something at once, rather than who owns a resource.
<code>CountDownLatch</code>	Settlement	One thread waits until N others finish. One-shot, so a new latch per batch.
<code>ScheduledExecutorService</code>	Settlement, metrics reporter	Periodic work. Better than a thread with <code>sleep</code> in a loop because it survives timing drift and is managed by the pool.
<code>AtomicLong</code> / <code>LongAdder</code>	Metrics	Lock-free counters. <code>LongAdder</code> beats <code>AtomicLong</code> when many threads increment the same counter.
<code>Interrupt</code> + <code>shutdown</code> / <code>awaitTermination</code>	Graceful shutdown	The only correct way to stop threads. Never <code>Thread.stop()</code> , never a <code>while(true)</code> with no exit.
Virtual threads	Final benchmark	Cheap threads for blocking I/O. Comparing them against the I/O pool shows when they help and when they don't.

Development flow

Ten phases (0 to 9), about 9–10 hours in total. Each phase leaves a running program and ends with a commit. Revise the listed topics right before the phase that needs them, not all at the start. If the day runs short, phases 1–6 are the core; 7–9 can wait for tomorrow.

1. **Phase 0 — Setup (20 min).** Maven project, Java 21, SLF4J + Logback, JUnit 5. Create the `Transaction`, `CheckResult` and `Decision` records. Add a `Main` that runs for a fixed duration.
2. **Phase 1 — Raw threads and your own buffer (1.5 h).** Build R1 (generator), R2 (`BoundedBuffer` with `wait/notifyAll`), and R3 v1 (4 raw worker threads that just sleep 20 ms to fake processing).
 - Revise first: thread states, `synchronized` and monitors, `wait` vs `sleep`, why `while` not `if`, interrupt handling.
 - Done when: it runs 10 s, the generator visibly slows when you make workers slower (backpressure), and Ctrl+C stops everything via interrupts.
3. **Phase 2 — Audit logger and metrics (45 min).** Build R7 (audit thread with `LinkedBlockingQueue` and poison pill) and R10 counters with `LongAdder`.
 - Revise first: `BlockingQueue` methods (`put / take` vs `offer / poll`), `atomics` vs `volatile`.
 - Done when: every processed transaction appears once in the audit file, and counters print every 2 s.
4. **Phase 3 — Executors (1 h).** Replace raw workers with a dispatcher plus a hand-constructed `ThreadPoolExecutor` (R3 v2). Write a named `ThreadFactory`.
 - Experiment: shrink the pool queue and try `AbortPolicy`, then `CallerRunsPolicy`; note what happens to the generator.
 - Revise first: core vs max pool size, how the queue decides when new threads are created, rejection policies.
5. **Phase 4 — Callables and plain Future (45 min).** Build R4 checks as `Callable`s. Submit all three, call `future.get()` on each, and log the per-transaction latency.
 - Done when: you have a latency number. Keep it; Phase 5 must beat it.
6. **Phase 5 — CompletableFuture pipeline (1.5 h).** Build R5: parallel checks on two pools, `allOf` combine, `orTimeout` on fraud, `exceptionally` fallback, `thenAccept` to ledger and audit.
 - Revise first: `thenApply` vs `thenCompose`, `allOf`, `handle` vs `exceptionally`, why you pass your own executor instead of using the common pool.
 - Done when: latency is close to the slowest single check rather than the sum, and forced timeouts show as `FRAUD_TIMEOUT` declines.
7. **Phase 6 — Executor decorators (1 h).** Build R11: `TracingExecutor`, `TimingExecutor`, `RetryingCallable`, stacked together.

- Revise first: the Decorator pattern; how MDC is a `ThreadLocal`, which is exactly why it vanishes on a pool thread.
 - Done when: every log line from a worker shows the transaction id, and balance-check failures are retried and counted.
- 8. Phase 7 — Ledger locks and issuer limit (1 h).** Build R6 (reproduce the transfer deadlock first, capture a thread dump with `jstack`, then fix it) and R8 (`Semaphore`).
- Revise first: the four deadlock conditions, `ReentrantLock` vs `synchronized`.
- 9. Phase 8 — Settlement (45 min).** Build R9 with a `ScheduledExecutorService` and a `CountDownLatch` per batch.
- Revise first: `scheduleAtFixedRate` vs `scheduleWithFixedDelay`; what happens if a scheduled task throws.
- 10. Phase 9 — Graceful shutdown (45 min).** Build R12 in the exact order listed there.
- Done when: the final check, `received = approved + declined`, passes on every run, and the audit file is complete.
- 11. Stretch — Compare and measure.** Swap your `BoundedBuffer` for `ArrayBlockingQueue` behind the same interface. Then run the I/O pool as `Executors.newVirtualThreadPerTaskExecutor()` and compare throughput at 50, 500 and 5,000 transactions per second.

Package structure and final checklist

```
com.g.txnsim
├─ Main.java           // wiring, run duration, shutdown hook
├─ model/              // Transaction, CheckResult, Decision
(records)
├─ ingest/            // TransactionGenerator, BoundedBuffer, Buffer
interface
├─ processing/        // Dispatcher, AuthorizationPipeline
├─ checks/            // FraudCheck, BalanceCheck, LimitCheck
├─ concurrent/        // TracingExecutor, TimingExecutor,
RetryingCallable, NamedThreadFactory
├─ ledger/            // AccountLedger, IssuerBankClient
├─ settlement/        // SettlementJob
├─ audit/             // AuditLogger
└─ metrics/           // Metrics, MetricsReporter
```

The project is done when all of these hold:

- ☐ No Spring, no AOP; only SLF4J, Logback and JUnit on the classpath
- ☐ `BoundedBuffer` uses only `synchronized`, `wait` and `notifyAll`
- ☐ Generator slows down when processors are slow (backpressure visible in metrics)
- ☐ Pipeline latency is near the slowest check, not the sum of all three
- ☐ Forced fraud timeouts produce `FRAUD_TIMEOUT` declines, never hangs
- ☐ Every worker log line carries the transaction id via `TracingExecutor`
- ☐ Deadlock reproduced, thread dump saved, then fixed with lock ordering
- ☐ Never more than 5 concurrent issuer calls
- ☐ Settlement report every 5 s with per-merchant totals
- ☐ On shutdown, received = approved + declined, and the audit file has every event
- ☐ One-page README explaining why each concept was used where it was